



V. M. Sabadosh, V. M. Kotsovsky

Uzhhorod National University, Uzhhorod, Ukraine

AUTOMATED DETECTION AND COMPARATIVE ANALYSIS OF SQL ANTIPATTERNS IN TEXT-TO-SQL DATASETS

This study examines the quality of SQL queries in Text-to-SQL datasets, focusing on antipatterns in gold-standard queries used to train and evaluate large language models (LLMs). While such queries may be syntactically valid and executable, they can still contain structural flaws that affect correctness, portability, and benchmark reliability. Since these flaws do not prevent successful execution, they often go undetected by conventional validation methods – creating what we call a correctness paradox.

A rule-based static analysis framework was designed and implemented for detecting 14 SQL antipatterns in Text-to-SQL datasets. The rules operate on abstract syntax trees produced by a dialect-aware SQL parser. Each detected antipattern is classified into one of four severity levels (Critical, High, Medium, Low) using a configurable profile for SQLite, PostgreSQL, and other RDBMSs.

The framework was applied to six partitions from three major Text-to-SQL benchmarks: Spider (11,840 queries), BIRD (10,962 queries), and the Gretel synthetic dataset (100,000 queries). Overall, 82.5–89.7 % of queries were antipattern-free, and 16,554 antipattern occurrences across 14 types were detected. Spider was dominated by Missing GROUP BY, BIRD by Function in WHERE, whereas Gretel exhibited the broadest defect spectrum and was the only dataset containing 16 syntactically invalid SQL queries. Manual verification of all Critical-severity cases confirmed that the vast majority represented genuine defects; a small number of borderline cases were retained due to the framework's deliberately conservative detection policy.

A comparative evaluation on a representative sample of 1,237 queries showed that, while our framework detected antipatterns across all 14 classes, the competing tool identified the expected antipattern in only 58.6 % of cases, covering just 4 classes. For Cartesian products specifically, the competing tool reached only 29.5 % recall, missing eight distinct syntactic patterns.

Based on these findings, we recommend integrating automated antipattern detection into benchmark quality assurance workflows, prioritizing and fixing Critical-severity cases across the analysed datasets, correcting the 16 syntactically invalid SQL queries identified in the Gretel dataset, and resolving Missing GROUP BY patterns that undermine cross-engine portability. To support adoption of these recommendations, the framework is released as an open-source tool.

Keywords: SQL query quality, classification, AST-based detection, benchmark datasets, LLM.

Introduction

Text-to-SQL systems [1], which transform natural-language questions into executable SQL queries, have advanced rapidly in recent years owing to progress in large language models (LLM) and benchmark-driven evaluation. Their practical relevance continues to grow, as natural-language interfaces lower the barrier to database access for users without specialized SQL expertise. In this context, benchmark datasets play a central role: they provide the supervision used for fine-tuning, the examples used in few-shot prompting, and the testbeds used to compare competing systems.

The quality of these datasets is therefore a methodological issue rather than a purely technical detail. A Text-to-SQL example typically consists of a natural-language question, a database schema, and a gold SQL query [2]. If the target SQL is flawed, the defect can be propagated into model training,

prompt conditioning, and evaluation. Even a relatively small share of problematic examples may introduce systematic bias, especially when the same benchmark is repeatedly reused across model development cycles.

Among the possible defects in gold SQL, antipatterns occupy a distinctive position. Unlike syntax errors, which prevent parsing, or execution errors, which prevent successful query evaluation, SQL antipatterns are structurally problematic queries that may still parse and execute correctly. Their risk lies in subtler properties: they may produce non-deterministic results, rely on dialect-specific behaviour, degrade execution efficiency, or encode unsafe query structure. As a result, conventional validation based only on successful parsing or execution is insufficient for assessing benchmark quality. A dataset may appear formally correct while still containing query patterns that reduce its reliability for robust model training and evaluation.

Despite the central role of datasets, much of the Text-to-SQL research has focused primarily on model architectures, prompting strategies, and leaderboard performance. By contrast, the quality of benchmark SQL itself has received much less systematic attention. Prior studies have examined benchmark composition, robustness, and semantic annotation errors, yet a reusable framework for large-scale SQL antipattern auditing and cross-benchmark comparison remains lacking. This gap is important because benchmark quality should be evaluated not only in terms of annotation coverage and executability, but also in terms of structural SQL quality.

This paper addresses that gap by focusing on automated SQL antipattern detection as a distinct dimension of benchmark quality assessment. It examines both human-curated and synthetic datasets to determine whether their SQL quality profiles differ in systematic ways. This comparison is particularly relevant because synthetic corpora are increasingly used to expand training data and may introduce risks that are not readily captured by conventional validation procedures.

The object of the study – is the quality of SQL queries in Text-to-SQL training and evaluation datasets.

The subject of the study – is the methods of automated detection and comparative analysis of SQL antipatterns in benchmark SQL queries.

The purpose of the study – is to develop a dialect-configurable framework for automated SQL antipattern detection and to apply it to a comparative analysis of major human-curated and synthetic Text-to-SQL datasets.

To achieve the stated purpose, the following *main research tasks* were identified:

- 1) to analyse recent studies on Text-to-SQL benchmark quality, robustness, and SQL query validation;
- 2) to design and implement a dialect-configurable framework for AST-based SQL antipattern detection with severity classification;
- 3) to conduct a comparative empirical analysis of SQL antipatterns across six partitions from three widely used Text-to-SQL datasets;
- 4) to identify benchmark-specific antipattern profiles and formulate practical recommendations for benchmark quality assurance.

Materials and Methods of Research. The study employed methods of static SQL analysis, abstract syntax tree analysis, rule-based pattern detection, comparative dataset analysis, and descriptive statistics. The software implementation was developed in Python using the `sqlglot` [3] dialect-aware SQL parser and DuckDB as the analytical database. The empirical material comprised six partitions from three major Text-to-SQL benchmarks: Spider [2], BIRD [4], and Gretel [5] synthetic Text-to-SQL. All analyses were conducted on an Apple M1 Pro MacBook Pro (32 GB RAM), with the 100,000-query Gretel corpus processed in under 7 minutes (~251 queries/s).

Analysis of Recent Research and Publications. The development of Text-to-SQL systems has been strongly

shaped by benchmark datasets. A major milestone was the release of Spider [2] at EMNLP 2018, which introduced a large-scale and cross-domain benchmark comprising 10,181 natural language questions mapped to 5,693 unique SQL queries across 200 databases from 138 domains. Spider remains one of the most widely used Text-to-SQL benchmarks and has served as the primary evaluation target for a large body of subsequent research.

The BIRD benchmark [4] extends this line of work toward more realistic and content-rich database settings. In contrast to earlier benchmarks, BIRD emphasizes large-scale databases, noisy values, and external knowledge dependencies, thereby reflecting a more practically challenging evaluation environment. The benchmark contains 12,751 question-SQL pairs across 95 databases spanning 37 professional domains. These characteristics make BIRD particularly relevant for assessing Text-to-SQL systems intended for real-world deployment.

In parallel, synthetic data generation has emerged as a scalable approach to alleviating the scarcity of high-quality supervised Text-to-SQL training data. One representative example is the Gretel synthetic Text-to-SQL dataset [5], which provides over 100,000 examples generated with the assistance of large language models and covers multiple business-oriented domains. Synthetic corpora of this kind are increasingly used to expand training resources, but their SQL quality characteristics remain insufficiently studied.

A growing body of evidence indicates that dataset quality directly affects the performance and robustness of Text-to-SQL systems. The Dr. Spider study [6] demonstrated that model performance degrades substantially under reduced data quality, with execution accuracy dropping by 14.0 % on average and by as much as 50.7 % under severe perturbations. Similarly, Arctic-Text2SQL [7] showed that adding lower-quality synthetic samples to training data can reduce final model accuracy even when the overall dataset size increases, highlighting that data quality may be more important than raw corpus scale. These findings motivate closer scrutiny of the SQL queries used as gold-standard training and evaluation targets.

Despite the central role of benchmark quality, existing validation efforts remain limited in scope. Mitsopoulou and Koutrika [8] analyze Text-to-SQL benchmarks in terms of structural, operator, and lexical characteristics, but their study is primarily descriptive and does not provide an automated diagnostic framework for SQL quality auditing. NL2SQL-BUGs [9] addresses another important aspect of dataset quality by identifying semantic annotation errors in gold SQL through expert analysis. This line of work is complementary to the present study, which focuses on SQL antipatterns as structurally problematic query formulations that may still parse, execute, and return plausible outputs while reducing portability, reliability, efficiency, or safety.

Several practical tools have been proposed for SQL quality checking in broader database contexts. `SqlCheck` [10] provides rule-based detection for a broad set of query-level SQL anti-patterns, including `SELECT *`, problematic `NULL`

usage, GROUP BY misuse, pattern-matching issues, unnecessary DISTINCT, nested subqueries, and selected join-related query smells. However, its published rule set covers only a subset of the antipattern categories central to the present study. More importantly, SqlCheck is designed as a general SQL anti-pattern checker rather than a framework for corpus-level auditing of Text-to-SQL benchmark datasets. SQLFluff [11] focuses mainly on formatting, style, and syntactic consistency, although it also provides selected convention rules. Taken together, these tools provide useful foundations, but they are not intended for severity-aware cross-benchmark analysis of SQL antipatterns in Text-to-SQL datasets.

Therefore, an important gap remains in the literature. Existing studies do not provide a reusable framework for large-scale SQL antipattern auditing across benchmark datasets. In particular, the contrast between human-curated and synthetic Text-to-SQL datasets has not been systematically examined from the perspective of SQL antipattern prevalence and risk profile. Addressing this gap requires a dedicated static-analysis framework capable of identifying structurally problematic SQL patterns at scale and supporting comparative analysis across benchmark families.

Research results and their discussion

Proposed framework for SQL antipattern detection. We designed and implemented *text2sql-antipattern-analyzer*, a

framework for systematic quality assessment of Text-to-SQL benchmarks. The framework is released as open-source software and implemented in Python using AST-based static analysis, with *sqlglot* [3] as a dialect-aware SQL parser and DuckDB as a columnar analytics backend. Its architecture comprises five runtime layers (Fig. 1), coordinated by a separate configuration and orchestration layer that declaratively defines system components and their parameters through YAML files, including SQL-dialect-specific antipattern severity settings (e. g., Missing GROUP BY is classified as High severity for SQLite but Critical for PostgreSQL, where it leads to execution failure).

The input layer abstracts heterogeneous data sources behind a common iterator interface, implementing loaders for JSONL (including gzip-compressed), CSV, JSON, and HuggingFace formats. All loaders yield records on demand, keeping memory usage constant regardless of dataset size.

The normalization layer standardizes records through alias mapping (reconciling field names such as 'query' vs 'sql', 'db_id' vs 'dbId') and unique identifier assignment for consistent cross-referencing.

The analysis layer constitutes the core of the framework and implements a two-stage pipeline. In the first stage, SQL statements are parsed into abstract syntax trees (ASTs) using *sqlglot* [3]. Queries that cannot be parsed are flagged and excluded from further antipattern analysis.

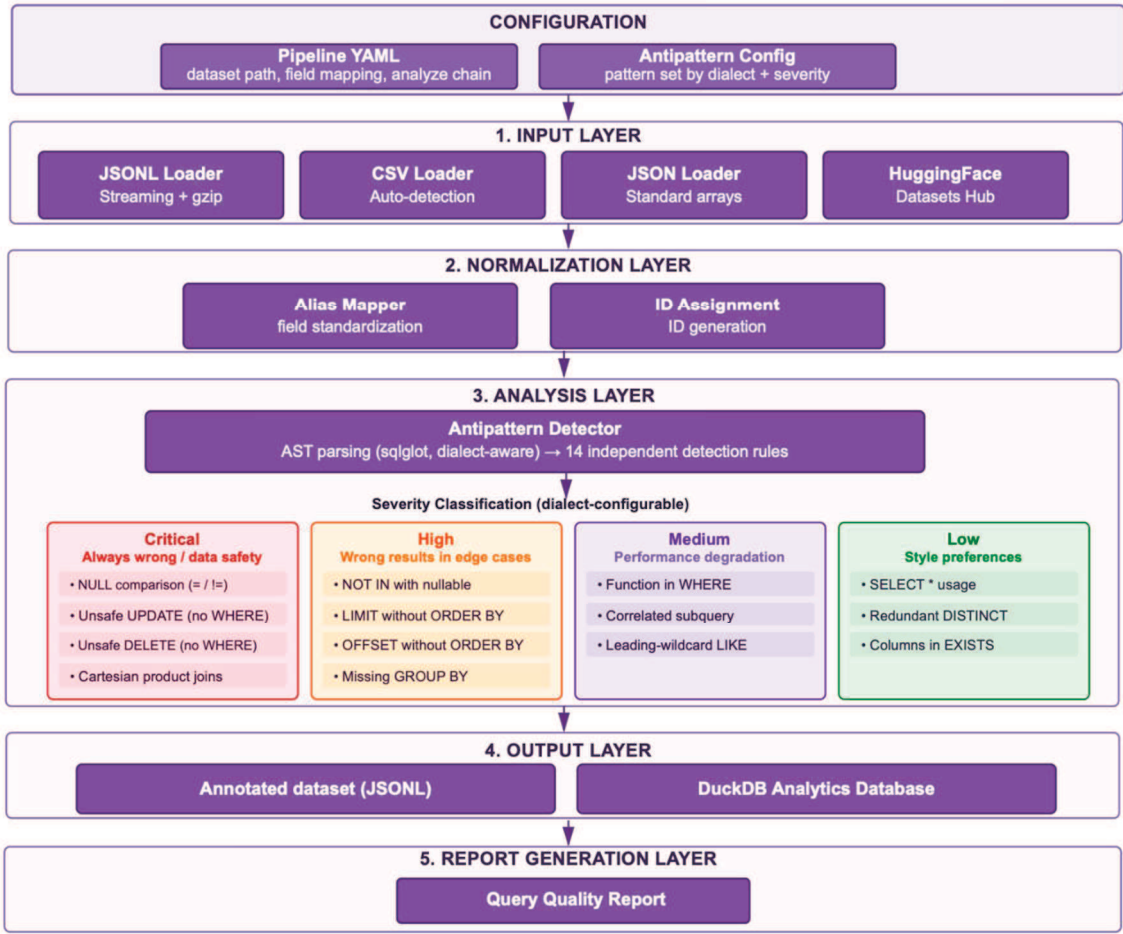


Fig. 1. Five-layer architecture of the Text2SQL Antipattern Analyzer

In the second stage, 14 independent detection rules traverse the AST and identify antipattern signatures through typed node matching (e. g., EQ, NEQ, AggFunc, UPDATE, DELETE, WHERE, JOIN, SubQuery, LIKE). Because the rules operate independently, multiple concurrent antipatterns can be detected within a single query. The detector identifies

14 distinct SQL antipattern types grouped into four severity levels. Although Unsafe UPDATE and Unsafe DELETE are detected by closely related logic, they are reported as separate antipattern types because of their different operational consequences. The taxonomy is based on established SQL quality literature [12–14].

Table 1. Summary of 14 detected SQL antipattern types by severity (SQLite configuration)

Severity	Antipattern	Impact
Critical	NULL comparison (col = NULL)	Always evaluates to NULL, breaking predicate logic [12, 13]
	Unsafe UPDATE without WHERE	Missing WHERE modifies all rows [12]
	Unsafe DELETE without WHERE	Missing WHERE deletes all rows [12]
	Cartesian product	Missing JOIN condition causes $N \times M$ row explosion [12]
High	NOT IN with nullable subquery	NULL causes the entire predicate to fail silently [12, 13]
	LIMIT without ORDER BY	Non-deterministic result subset [12]
	OFFSET without ORDER BY	Non-deterministic pagination [12]
	Missing GROUP BY	Bare columns produce arbitrary values in SQLite and violate standard SQL portability [14, 15]
Medium	Functions in WHERE	Prevents index usage, forces full scan [12]
	Correlated subqueries	$O(n^2)$ row-by-row execution in non-decorrelated cases [12]
	Leading wildcard LIKE	Prevents index prefix matching [12]
Low	SELECT *	Obscures column dependencies [12]
	Redundant DISTINCT	Unnecessary duplicate elimination [12]
	Columns in EXISTS subquery	Wasteful column retrieval [12]

Despite its historical name, the Function in WHERE rule captures an important subset of non-SARGable predicate forms [16]. Beyond standard function calls such as UPPER(col), DATE(col), and COALESCE(col, ...), it also detects explicit type conversions (e.g., CAST(col AS ...)) and arithmetic expressions applied to columns in WHERE and HAVING clauses, such as col + 1, col * 2, or col || 'x'. Such transformations typically prevent efficient use of ordinary B-tree indexes on the affected column and often result in scan-based access paths unless the DBMS can exploit a functional or expression index. By contrast, functions and expressions applied exclusively to literals (e.g., name = UPPER('John')) are excluded, since they do not reduce index eligibility for the indexed column.

The proposed severity stratification reflects the practical consequences of the detected patterns. Critical-level antipatterns are associated with correctness violations or potentially destructive data modification. High-severity antipatterns may still execute successfully but can produce non-deterministic or logically unsafe results. Medium-severity patterns mainly affect execution efficiency, whereas Low-severity patterns concern code clarity, maintainability, and stylistic quality. This structured taxonomy is used throughout the subsequent analysis to compare antipattern prevalence across benchmark families.

The detection engine is validated by 333 tests covering all implemented rules under the SQLite configuration, including representative edge cases. The test suite was systematically constructed during empirical analysis of the three investigated datasets. As part of framework validation, detected antipattern cases across Spider, BIRD, and Gretel were manually reviewed for false positives and false negatives; queries confirmed as genuine defects or false-positive triggers were subsequently incorporated as dedicated regression test cases.

The output layer stores results through two parallel streams: an annotated JSONL dataset preserving provenance, and a DuckDB columnar analytics database enabling ad-hoc SQL queries and Parquet export.

The report generation layer converts stored metrics into human-readable Query Quality Report that aggregates antipattern statistics by type, severity, and dataset partition. Reports can be regenerated without re-running the analysis pipeline.

Comparative analysis results across benchmarks. The study covers six partitions from three major Text-to-SQL benchmarks (Table 2): Spider 1.0 from the Yale LILY repository, BIRD from the official benchmark repository, and Gretel synthetic data from the Hugging Face Datasets Hub. In total, the study includes 122,802 queries. The empirical analysis

was conducted using the proposed framework, with all analyses performed under the SQLite-specific severity configuration corresponding to the original database format of each dataset.

Tables 3 and 4 report antipattern frequencies by type across dataset partitions and reveal distinct benchmark signatures. Spider is characterized primarily by Missing

GROUP BY occurrences, with a notable prevalence of NOT IN with nullable subquery cases, while BIRD is most strongly associated with Function in WHERE. Gretel exhibits the broadest defect taxonomy, covering all 14 detected antipattern types and substantially larger absolute counts than the human-curated benchmarks.

Table 2. Datasets under analysis

Dataset	Type	Queries	Sources
Spider Dev	Human-curated	1,034	[2]
Spider Test	Human-curated	2,147	[2]
Spider Train	Human-curated	8,659	[2]
BIRD Dev	Human-curated	1,534	[4]
BIRD Train	Human-curated	9,428	[4]
Gretel Train	Synthetic	100,000	[5]
Total		122,802	

Table 3. Antipattern distribution across Spider dataset partitions

Antipattern	Severity	Spider Dev	Spider Test	Spider Train
Cartesian product	critical	2 (0.3 %)	6 (0.2 %)	13 (0.2 %)
Missing GROUP BY	high	106 (10.3 %)	239 (11.1 %)	600 (6.9 %)
NOT IN with nullable subquery		46 (4.4 %)	74 (3.4 %)	228 (2.6 %)
Function in WHERE	edium	–	–	6 (0.1 %)
Leading wildcard LIKE		12 (1.2 %)	50 (2.3 %)	140 (1.6 %)
Correlated subquery		–	–	–
SELECT *	Low	3 (0.3 %)	12 (0.6 %)	62 (0.7 %)
Redundant DISTINCT		–	4 (0.3 %)	131 (1.5 %)
Columns in EXISTS subquery		–	–	–
Total occurrences		169	385	1,180
Distinct antipattern types		5	6	7

Table 4. Antipattern distribution across BIRD and Gretel dataset partitions

Antipattern	Severity	BIRD Dev	BIRD Train	Gretel Train
Cartesian product	Critical	1 (0.1 %)	17 (0.2 %)	329 (0.3 %)
NULL comparison		–	4 (< 0.1 %)	1 (< 0.1 %)
Unsafe UPDATE (no WHERE)		–	–	54 (0.1 %)
Unsafe DELETE (no WHERE)		–	–	47 (< 0.1 %)
Missing GROUP BY	High	20 (1.4 %)	240 (2.6 %)	1,933 (1.9 %)
NOT IN with nullable subquery		2 (0.1 %)	14 (0.1 %)	506 (0.5 %)
LIMIT without ORDER BY		6 (0.4 %)	123 (1.3 %)	134 (0.1 %)
OFFSET without ORDER BY		–	–	1 (< 0.1 %)
Function in WHERE	Medium	144 (9.4 %)	436 (4.6 %)	5,926 (5.9 %)
Leading wildcard LIKE		19 (1.2 %)	144 (1.5 %)	1,897 (1.9 %)
Correlated subquery		2 (0.1 %)	2 (< 0.1 %)	389 (0.4 %)
SELECT *	Low	–	9 (0.1 %)	2,317 (2.3 %)
Redundant DISTINCT		3 (0.4 %)	24 (0.3 %)	46 (0.1 %)
Columns in EXISTS subquery		–	–	30 (< 0.1 %)
Total occurrences		197	1,013	13,610
Distinct antipattern types		8	10	14

Table 5 complements this view by aggregating findings by severity level. Spider is dominated by High-severity defects, BIRD by Medium-severity defects, and Gretel shows the widest severity spread together with the highest total number

of occurrences. Importantly, only Gretel contains destructive DML antipatterns (Unsafe UPDATE / DELETE without WHERE), indicating not only quantitative but also qualitative differences in operational risk between benchmark families.

Table 5. Antipattern occurrences by severity level

Severity	Spider Dev	Spider Test	Spider Train	BIRD Dev	BIRD Train	Gretel Train
Critical	2	6	13	1	21	431
High	152	313	828	28	377	2,574
Medium	12	50	146	165	582	8,212
Low	3	16	193	3	33	2,393
Total	169	385	1,180	197	1,013	13,610

Table 6. Datasets under analysis and overall query quality metrics

Dataset	Queries w/o Antipatterns	Clean %	Queries with syntax errors
Spider Dev	865	83.7 %	0
Spider Test	1,772	82.5 %	0
Spider Train	7,496	86.6 %	0
BIRD Dev	1,338	87.2 %	0
BIRD Train	8,454	89.7 %	0
Gretel Train	87,036	87.1 %	16
Total	106,961	–	16

Table 7. Categories of syntactically invalid SQL queries identified in Gretel

Error category	Count
Multiple WHERE clauses	8
Multiple GROUP BY clauses	4
Incomplete INSERT ... VALUES statement	2
Parenthesis / syntax break	1
Invalid expression / unexpected token	1
Total	16

A complementary perspective is provided by the overall quality metrics shown in Table 6. The proportion of antipattern-free queries ranges from 82.5 % in Spider Test to 89.7 % in BIRD Train. Spider and BIRD achieved full parse success in the analysis pipeline, whereas the Gretel corpus additionally contained syntactically invalid SQL queries that could not be analysed at the AST stage.

To further clarify the quality profile of Gretel, Table 7 groups the syntactically invalid queries by error category. These cases represent a separate class of benchmark defects because, unlike antipatterns, they fail already at the parsing stage and therefore cannot be treated as structurally flawed but executable SQL. Their presence further distinguishes the synthetic benchmark from the human-curated datasets and strengthens the conclusion that synthetic generation pipelines require additional safeguards at both the parsing and structural validation stages.

Comparative validation against SqlCheck. To position the proposed framework relative to an existing tool, we conducted a comparative evaluation against SqlCheck [10], a widely used rule-based SQL anti-pattern checker. However, SqlCheck is intended for query-level linting and does not provide corpus-level quality auditing capabilities; accordingly, in this study it is used as a per-query comparative baseline rather than as an end-to-end benchmark validator. A stratified sample of 1,237 queries was drawn from all six dataset partitions: the sample included every query flagged with a Critical-severity antipattern (476 queries), all 600 Missing GROUP BY instances from Spider Train, and up to 20 randomly selected queries for each of the remaining 9 non-critical

antipattern classes, limited by availability (161 queries in total). Each sampled query was analysed by SqlCheck v1.3 (executed in Docker, verbose mode, all risk levels: -v -r 1). All comparison verdicts were independently verified through manual inspection by the first author. To prioritise ambiguous cases for review, automated pre-screening was performed using Claude Opus 4.6 (Anthropic, 2026); all flagged cases were subsequently confirmed or rejected through human judgement. Table 8 summarises the per-class detection rates for the four antipattern classes in which at least one tool produced a non-zero result. Overall, SqlCheck detected the expected antipattern in 725 out of 1,237 sampled queries (58.6 %); the remaining 243 queries belonged to antipattern classes for which SqlCheck produced no output. Full agreement was observed for two classes: NULL comparison and SELECT *. Missing GROUP BY achieved near-full agreement (591 of 600, 98.5 %); the 9 missed queries all lacked a GROUP BY clause entirely, mixing aggregate functions with bare columns – a pattern SqlCheck does not check because its “GROUP BY Usage” rule fires only when the clause is present. SqlCheck had no coverage for 10 of the 14 antipattern classes examined in this study, including all Unsafe UPDATE / DELETE patterns. The largest source of disagreement was the Cartesian product class, where SqlCheck detected 109 of 369 instances (29.5 %).

To understand why SqlCheck missed 260 Cartesian-product instances, we manually classified every missed query by the syntactic join pattern involved. SqlCheck’s detection relies primarily on the rule “JOIN Without Equality Check”, which fires only when a JOIN keyword appears without any ON clause. This single heuristic is

insufficient for the eight distinct Cartesian-producing patterns identified in the sample (Table 9).

The dominant category is comma-join (46.2 %), where two or more tables appear in the FROM clause separated by commas without a JOIN keyword or a linking WHERE predicate. The second-largest category (16.5 %) involves tautological ON predicates such as ON 1=1 or ON TRUE, which SqlCheck does not recognise because the ON clause is syntactically present. A further 13.8 % use the ON clause as a filter predicate (e.g., ON employees.position = 'Operator') rather than as a table-linking condition. The remaining five categories account for 23.7 % and include explicit CROSS JOIN, self-referencing ON conditions, non-equality ON predicates, wrong-table references, and missing ON clauses.

For transparency, we note that the Cartesian product count reported by the proposed framework includes a small number of borderline cases where the JOIN keyword is present without an explicit ON clause, but the table-linking predicate is placed in the WHERE clause instead. Consider the following query from BIRD Train (item 7292):

```
SELECT T1.Description
```

```
FROM ProductDescription AS T1 INNER
JOIN Product AS T2
WHERE T2.Name = 'Headset...' AND
T1.productDescriptionID = T2.ProductID
```

Here, the INNER JOIN lacks an ON clause, and the inter-table predicate (T1.productDescriptionID = T2.ProductID) appears in WHERE. While the query may produce correct results for many SQL engines that treat this as an implicit equi-join, we intentionally classify it as a Cartesian-style risk pattern because (a) the SQL standard requires ON for INNER JOIN, (b) missing ON is a known source of accidental cross joins when queries are modified, and (c) this formulation is indistinguishable from a genuine Cartesian product at the AST level. Similar cases occur in Spider Test (items 508–511) and in Gretel, where queries use ON TRUE / ON 1=1 while moving the actual linking predicate to WHERE. These borderline instances represent fewer than 2 % of all detected Cartesian cases and reflect the framework’s conservative design choice: flagging structurally ambiguous patterns rather than silently accepting them.

Table 8. Per-class detection comparison between the proposed framework and SqlCheck

Antipattern	Severity	N	SQLCheck	%
NULL comparison	Critical	5	5	100.0
Cartesian product	Critical	369	109	29.5
Missing GROUP BY	High	600	591	98.5
SELECT *	Low	20	20	100.0
Total (shown classes)		994	725	–

Table 9. Classification of 260 Cartesian-product instances missed by SqlCheck

Join Pattern	Count	%
Comma-join (FROM t1, t2)	120	46.2
ON 1=1 / ON TRUE / ON FALSE	43	16.5
ON with filter predicate (literal, subquery, function)	36	13.8
Explicit CROSS JOIN	19	7.3
Self-referencing ON (ON T2.a = T2.b)	15	5.8
ON with non-equality / range predicate	12	4.6
ON referencing wrong / missing table	12	4.6
JOIN without ON clause	3	1.2

Discussion of research results. The results show that SQL quality in Text-to-SQL benchmarks cannot be assessed solely through parsing success and executable correctness. Across the analysed benchmark families, structural defects vary not only in frequency, but also in type, severity, and associated risks. This indicates that structural SQL quality should be treated as an independent dimension of benchmark assessment.

The observed differences between human-curated and synthetic datasets are particularly important. Human-curated benchmarks more often exhibit portability-sensitive and performance-related structural issues, whereas the synthetic Gretel dataset combines a broader defect spectrum with syntax-level failures and potentially destructive DML patterns. This suggests that synthetic data generation pipelines require stronger safeguards, including parsing-stage validation, structural rule-based auditing, and safety filtering for potentially harmful query forms.

The comparative evaluation against SqlCheck highlights that the advantage of the proposed framework lies not only in its design for benchmark-level assessment of Text-to-SQL datasets, but also in its broader antipattern coverage and higher accuracy in detecting structurally diverse defect patterns. While SqlCheck showed near-complete agreement for only a small subset of well-established rule categories, many antipattern classes examined in this study fell outside its rule set, including all Critical-severity unsafe UPDATE/DELETE patterns. For Cartesian products, in particular, SqlCheck relied on a single syntactic heuristic and therefore missed many valid join-defect forms captured by the proposed framework. These results show that general-purpose SQL linters are not sufficient as standalone tools for comprehensive structural quality assessment of Text-to-SQL benchmark datasets.

Scientific novelty of the obtained results – for the first time:

- a dialect-configurable AST-based framework for corpus-level auditing of gold SQL queries in Text-to-

SQL datasets has been designed and implemented, combining 14 operationalized antipattern detectors with severity profiles for different RDBMSs;

- an empirical analysis of SQL antipattern prevalence and distribution has been conducted across three widely used Text-to-SQL benchmarks – Spider, BIRD, and Gretel – revealing characteristic defect patterns specific to human-curated and synthetic datasets and establishing a quantitative baseline for benchmark quality assessment

Practical significance of the research results – the proposed framework can be directly integrated into benchmark curation workflows, synthetic Text-to-SQL data generation pipelines, and dataset validation procedures. Its application enables automatic identification of structurally risky and syntactically invalid SQL queries, prioritisation of remediation by severity level, and generation of reproducible audit reports for benchmark maintainers and researchers. The framework is open-source, processes large datasets quickly, and can be used in research and engineering practice. Its practical value is further supported by the independent manual verification of all found in studies 476 critical-severity cases, which confirmed them as genuine problematic queries, as well as by the identification of 16 syntactically invalid SQL queries and the high prevalence of Missing GROUP BY patterns, which together indicate clear priorities for benchmark remediation.

A key limitation of the proposed framework is its exclusive reliance on AST-level static analysis, without integration of schema-level metadata such as column types, nullability constraints, indexes, partitioning, or execution-plan evidence. As a result, the framework remains schema-agnostic and cannot reliably resolve certain cases whose interpretation depends on schema information. This limitation is directly relevant to patterns such as NOT IN with nullable subquery, where risk assessment depends on whether the subquery expression is in fact nullable, and to SARGability-related analysis, where implicit type conversions inferred from column metadata cannot be detected in the current mode. More broadly, the schema-agnostic nature of the framework also constrains the severity model itself, including automatic severity calibration. The current classification is intentionally single-axis and serves as a practical prioritisation scheme for dataset quality assurance, in which correctness-breaking defects (Critical/High) are distinguished from patterns whose impact is predominantly performance- or maintainability-related (Medium/Low). It does not, however, explicitly represent correctness risk and performance risk as independent dimensions. For example, SELECT * is classified by default as a low-severity structural antipattern, although its impact may increase when the projection includes large or resource-intensive columns. Although no JSONB, VARBINARY, or BYTEA columns were found in the analysed datasets, the presence of BLOB and JSON-typed columns suggests that schema-aware severity calibration could be beneficial even within the examined corpus. Accordingly, extending the framework toward schema-aware detection, severity calibration, and a multi-dimensional risk model remains an important direction for future work.

Conclusions

This study developed a dialect-configurable framework for automated detection of SQL antipatterns in Text-to-SQL datasets and applied it to a comparative analysis of the Spider, BIRD, and Gretel benchmarks. The results show that structural SQL quality should be treated as a separate dimension of benchmark evaluation alongside syntax and execution correctness.

The analysis revealed benchmark-specific differences in antipattern prevalence and identified critical issues that provide a practical roadmap for benchmark maintainers and synthetic Text-to-SQL pipeline developers. The comparative evaluation against SqlCheck confirmed the broader coverage and practical value of the proposed framework.

The framework is publicly available as the open-source project `text2sql-antipattern-analyzer` and can be used for automated structural quality auditing of SQL queries in benchmark curation and validation workflows. Its modular architecture also provides a foundation for future improvements through schema-aware detection, context-sensitive severity calibration, and a multi-dimensional risk model that better reflects distinct correctness and performance risks, thereby supporting further development toward a more comprehensive benchmark quality assurance pipeline.

To support transparency and reproducibility, the extended empirical results are provided in the project repository root folder *UJIT_Main_Article_Results*.

References

1. Shi, L., Tang, Z., Zhang, N., Zhang, X., Yang, Z. (2025). A Survey on Employing Large Language Models for Text-to-SQL Tasks. *ACM Computing Surveys*. <https://doi.org/10.1145/3737873>
2. Yu, T., Zhang, R., Yang, K., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 3911–3921. <https://doi.org/10.18653/v1/D18-1425>
3. Mao, T. (2025). SQLGlot: Python SQL Parser and Transpiler [Software]. GitHub. Available at: <https://github.com/tobymao/sqlglot>
4. Li, J., Hui, B., Qu, G., et al. (2023). Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. *Advances in Neural Information Processing Systems*, 36. <https://doi.org/10.48550/arXiv.2305.03111>
5. Gretel.ai (2024). Synthetic Text-to-SQL Dataset (`gretelai/synthetic_text_to_sql`). Hugging Face Datasets. Available at: https://huggingface.co/datasets/gretelai/synthetic_text_to_sql
6. Chang, S., Wang, J., Dong, M., et al. (2023). Dr. Spider: A Diagnostic Evaluation Benchmark towards Text-to-SQL Robustness. *Proceedings of the International Conference on Learning Representations*. Available at: <https://doi.org/10.48550/arXiv.2301.08881>
7. Yao, Z., Borchmann, L., Zhai, B., Zhang, H., He, Y. (2025). Arctic-Text2SQL-R1: Simple Rewards, Strong Reasoning in Text-to-SQL. *CoRR*, abs/2505.20315. <https://doi.org/10.48550/arXiv.2505.20315>

8. Mitsopoulou, A., Koutrika, G. (2025). Analysis of Text-to-SQL Benchmarks: Limitations, Challenges and Opportunities. Proceedings of the 28th International Conference on Extending Database Technology, 199–212. <https://doi.org/10.48786/EDBT.2025.16>
9. Liu, X., Shen, S., Li, B., Tang, N., Luo, Y. (2025). NL2SQL-BUGs: A Benchmark for Detecting Semantic Errors in NL2SQL Translation. Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 5662–5673. <https://doi.org/10.1145/3711896.3737427>
10. Dorn, T., Jarulraj, A. (2019). SqlCheck: Automatically Detecting Common SQL Anti-Patterns [Software]. GitHub. Available at: <https://github.com/jarulraj/sqlcheck>
11. SQLFluff Contributors. (2025). SQLFluff: The SQL Linter for Humans [Software]. Available at: <https://sqlfluff.com/>
12. Karwin, B. (2010). SQL Antipatterns: Avoiding the Pitfalls of Database Programming. Pragmatic Bookshelf.
13. Date, C. J. (2012). SQL and Relational Theory: How to Write Accurate SQL Code. 2nd ed. O'Reilly Media.
14. International Organization for Standardization. (2016). ISO/IEC 9075-2:2016 Information technology – Database languages – SQL – Part 2: Foundation (SQL/Foundation). Available at: <https://www.iso.org/standard/63556.html>
15. SQLite. (2025). SELECT – Bare columns in an aggregate query. Available at: https://sqlite.org/lang_select.html#bare_columns_in_an_aggregate_query

В. М. Сабадош, В. М. Коцовський

Ужгородський національний університет, м. Ужгород, Україна

АВТОМАТИЗОВАНЕ ВИЯВЛЕННЯ ТА ПОРІВНЯЛЬНИЙ АНАЛІЗ SQL-АНТИПАТЕРНІВ У НАБОРАХ ДАНИХ TEXT-TO-SQL

У роботі досліджено якість SQL-запитів у наборах даних Text-to-SQL, зокрема антипатерни в еталонних запитах, використовуваних для навчання та оцінювання великих мовних моделей. Хоча такі запити можуть бути синтаксично коректними, іноді вони містять структурні дефекти, які негативно впливають на правильність результатів, сумісність із різними СУБД і надійність бенчмарків. Оскільки ці дефекти не перешкоджають успішному виконанню запитів, традиційні методи валідації часто їх не виявляють.

Для виявлення 14 типів SQL-антипатернів спроектовано та реалізовано rule-based фреймворк статичного аналізу, орієнтований на перевірку SQL-запитів у наборах даних Text-to-SQL. Його правила працюють із абстрактними синтаксичними деревами, побудованими за допомогою діалектно-орієнтованого SQL-парсера, що дає змогу систематично аналізувати великі колекції запитів незалежно від їхнього походження. Кожен виявлений антипатерн класифікують за одним із чотирьох рівнів критичності (Critical, High, Medium, Low) відповідно до налаштованого профілю, який підтримує SQLite, PostgreSQL та інші реляційні СУБД.

Фреймворк застосовано до шести розподілів трьох основних Text-to-SQL бенчмарків: Spider (11 840 запитів), BIRD (10 962 запити) та синтетичного набору даних Gretel (100 000 запитів). Загалом 82,5–89,7 % запитів не містили антипатернів, тоді як сумарно виявлено 16 554 випадки антипатернів 14 різних типів. Для Spider найхарактернішим виявився антипатерн Missing GROUP BY, для BIRD – Function in WHERE, тоді як Gretel продемонстрував найширший спектр дефектів і став єдиним набором даних, у якому було виявлено 16 синтаксично некоректних SQL-запитів. Перевірка усіх випадків із рівнем Critical вручну підтвердила, що переважна більшість із них – справжні дефекти; невелику кількість спірних випадків було збережено через свідомо консервативну політику виявлення, закладену у фреймворк.

Порівняльне оцінювання на репрезентативній вибірці з 1 237 запитів показало, що хоча запропонований фреймворк виявляв антипатерни в усіх 14 класах, конкурентний інструмент правильно ідентифікував очікуваний антипатерн лише у 58,6 % випадків і охоплював тільки чотири класи.

На підставі отриманих результатів рекомендовано інтегрувати автоматизоване виявлення антипатернів у процеси контролю якості бенчмарків. Першочергову увагу потрібно приділити виправленню антипатернів рівня Critical у досліджуваних наборах даних, коригуванню 16 синтаксично некоректних SQL-запитів у наборі даних Gretel, а також виправленню антипатернів типу Missing GROUP BY, що знижують сумісність запитів між різними СУБД. Щоб забезпечити практичне впровадження цих рекомендацій, фреймворк опубліковано у відкритому доступі.

Ключові слова: якість SQL-запитів, класифікація, AST-орієнтоване виявлення, еталонні набори даних, великі мовні моделі.

Інформація про авторів:

Сабадош Володимир Мирославович, аспірант, кафедра інформаційно-керуючих систем та технологій.

Email: volodymyr.sabadosh@uzhnu.edu.ua; <https://orcid.org/0009-0006-9933-444X>

Коцовський Владислав Миронович, канд. техн. наук, доцент, кафедра інформаційно-керуючих систем та технологій.

Email: vladyslav.kotsovsky@uzhnu.edu.ua; <https://orcid.org/0000-0002-7045-7933>

Цитування за ДСТУ: Сабадош В. М., Коцовський В. М. Автоматизоване виявлення та порівняльний аналіз SQL-антипатернів у наборах даних Text-to-SQL. *Український журнал інформаційних технологій*. 2026, т. 8, № 1. С. 135–143.

Citation APA: Sabadosh, V. M., & Kotsovsky, V. M. (2026). Automated detection and comparative analysis of SQL antipatterns in Text-to-SQL datasets. *Ukrainian Journal of Information Technology*, 8(1), 135–143. <https://doi.org/10.23939/ujit2026.01.135>